Summary of Analysis

This analysis focused on predicting consumer behavior by evaluating various regression and tree-based models. Key efforts were dedicated to data cleaning, including handling missing data using median and mode replacements, dummy coding categorical variables, and renaming columns to reduce errors. Feature selection was refined through stepwise regression and VIF checks, ultimately narrowing down to 13 variables for initial modeling.

Regression models (linear, polynomial, and ridge) showed limited performance due to multicollinearity and optimization challenges, resulting in high RMSE. Tree-based models, particularly tuned Boosting and XGBoost, outperformed regression models by effectively managing categorical data and multicollinearity. Hyperparameter tuning and variable reduction to five key predictors, which are strongly linked to consumer behavior, significantly enhanced XGBoost's performance.

The final model highlights the strength of XGBoost with five consumer-focused variables, making it the most effective predictor. Further improvements could be achieved by experimenting with advanced models like SVM and neural networks and refining hyperparameters.

1. Introduction

- Description

In today's fast-paced digital environment, the success of a business heavily relies on consumer engagement with their online platforms, particularly measured through website visits. A critical determinant of this engagement is the Click-Through Rate (CTR) on digital advertisements. CTR reflects the effectiveness of an advertisement in capturing consumer interest and driving traffic.

- Objective

The primary objective of this analysis is to identify the most accurate prediction model for CTR based on a dataset comprising advertising performance metrics. By leveraging predictive analytics, the study aims to provide insights that can optimize advertising strategies and improve consumer engagement.

2. Variables

- 1 Dependent Variable : Click-Through Rate(CTR) -the percentage of users who clicked on the ad after viewing it.
- 28 variables representing customer behavior and ad characteristics:
- Examples: Targeting Score, Visual Appeal, Contextual Relevance, CTA Strength, Position on Page, Day & Time.
- Includes 20 numeric and 8 categorical variables.

3. Hyphotesis

- Null Hypothesis (H_0): There is no significant difference between the CTR (Click-Through Rate) data across the groups or conditions being tested.
- Alternative Hypothesis (H_1): There is a significant difference between the CTR (Click-Through Rate) data across the groups or conditions being tested.

4. Data Exploration

```
# Explore dataset summary
summary(analysis)

# Identify missing values
missing_values <- colSums(is.na(analysis))
print(missing_values)
```

5. Data Preparation

- Imputation:
    - Initially attempted to use `drop_na` to remove rows with null values, but this approach failed due to the requirement of retaining at least 1000 rows for submission.
    - Resorted to imputing missing values where necessary:
        - Median values for numeric variables.
        - Mode for categorical variables.
- Generating Dummy Variables:

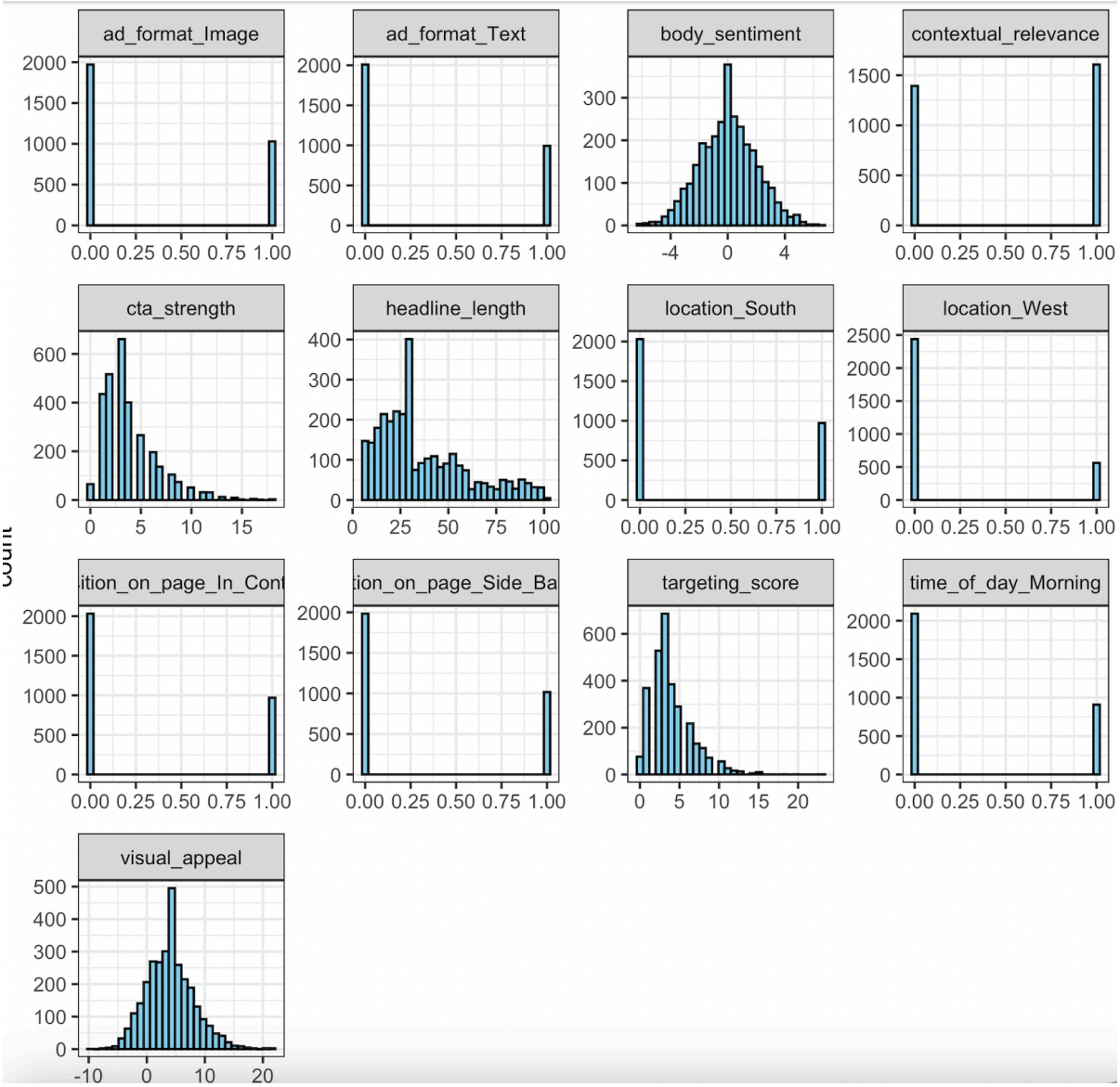
```
library(fastDummies)
analysis <- dummy_cols(analysis, select_columns = c("ad_format", "age_group", "day_of_we
ek", "location", "gender", "device_type", "position_on_page", "time_of_day"),remove_firs
t_dummy = TRUE)
analysis <- analysis %>% select(-c("ad_format", "age_group", "day_of_week", "locatio
n","gender", "device_type", "position_on_page", "time_of_day"))
```

- Column Name Formatting:

```
colnames(analysis) <- make.names(colnames(analysis))
```

- Exploratory Data Visualizations

The following plot shows the distributions of selected predictors:


ad_format_Image
ad_format_Text
body_sentiment
contextual_relevance
cta_strength
headline_length
location_South
location_West
ition_on_page_In_Cont
tion_on_page_Side_Ba
targeting_score
time_of_day_Morning
visual_appeal
count

```
library(ggplot2)
library(reshape2)
library(dplyr)

# select variables
selected_vars <- c("ad_format_Image", "ad_format_Text", "body_sentiment", "contextual_re
levance",
                   "cta_strength", "headline_length", "location_South", "location_West",
                   "position_on_page_In_Content", "position_on_page_Side_Bar", "targetin
g_score",
                   "time_of_day_Morning", "visual_appeal")


df_selected <- analysis %>% select(all_of(selected_vars))

# long format
df_long <- melt(df_selected)

ggplot(df_long, aes(x = value)) +
  geom_histogram(bins = 30, fill = "skyblue", color = "black") +
  facet_wrap(~variable, scales = "free", ncol = 4) +
  theme_minimal() +
  labs(title = "Distribution of Key Variables")
```

- Multicollinearity Check:
    - Pairwise correlations computed, and variables with correlation > 0.85 were removed to address redundancy.

```
cor_matrix <- cor(analysis %>% select_if(is.numeric))
high_cor <- findCorrelation(cor_matrix, cutoff = 0.85)
analysis <- analysis %>% select(-names(high_cor))
```

6. Data Splitting

-Data Splitting - Split the analysis dataset into training and testing sets to evaluate model performance:

```
library(caret)
set.seed(1031)
split <- createDataPartition(y = analysis$CTR, p = 0.7, list = FALSE)
train <- analysis[split, ]
test <- analysis[-split, ]
```

7. Model Testing

- To identify the best-performing model based on RMSE comparison, I configured the tree models with cv=3 and ntree=100.
- A. Hybrid Stepwise Variable Selection:
    - Selected 13 variables using stepwise regression.
    - Variables tested with consistent conditions across models.

- B. Baseline Linear Regression:
    - The results indicated that the model had a moderate explanatory power with an R squared of 0.42; however, it exhibited a high RMSE on the Kaggle dataset, suggesting potential room for improvement.

```
lm_model <- lm(CTR ~ ., data = train)
```

- Therefore, I applied polynomial regression using 2-3 important variables identified through the summary of the lm_model. However, the results showed that while the model achieved an R squared of 0.42, indicating moderate explanatory power, it also had the highest RMSE, suggesting it was less effective in prediction accuracy.
- C. Decision Trees and Tuned Trees:
    - While exploring tree models, the RMSE decreased noticeably, indicating improved predictive performance. I assumed that the data likely contains a significant number of qualitative variables serving as important predictors. However, it is worth noting that tree models typically do not achieve the same level of accuracy as linear regression for datasets with strong linear relationships.

```
library(rpart)
tree <- rpart(CTR ~ ., data = train, method = "anova")

library(caret)
trControl <- trainControl(method = "cv", number = 3)
tuneGrid <- expand.grid(cp = seq(0.001, 0.1, 0.001))
tuned_tree <- train(CTR ~ ., data = train, method = "rpart", trControl = trControl, tune
Grid = tuneGrid)
```

- D. Random Forest and Bagging
    - To enhance the performance of the tree model, I explored ensemble techniques: bagging, Random Forest, and boosting.
    - Bagging: Implemented bootstrapping (repeated sampling from the same dataset) followed by averaging to improve accuracy by reducing variance.
    - Random Forest: Since bagging can overemphasize a few strong predictors when variables are highly correlated, I employed Random Forest, which introduces randomness by selecting a subset of predictors at each split, thereby mitigating this bias.
    - Boosting: Boosting iteratively improves model performance by learning from errors of previous models and adjusting sequentially. By tuning hyperparameters, this method can significantly increase accuracy.

```
library(randomForest)
rf_model <- randomForest(CTR ~ ., data = train, ntree = 100)
rf_predictions <- predict(rf_model, test)
rf_rmse <- sqrt(mean((rf_predictions - test$CTR)^2))

library(ipred)
bag_model <- bagging(CTR ~ ., data = train, nbagg = 100)
```

- E. Gradient Boosting (GBM):

- The tuned boosting model has a higher likelihood of achieving the lowest RMSE. By adjusting the shrinkage parameter, I controlled the learning rate of the boosting algorithm, ensuring a more gradual and effective improvement in model performance.

```
library(gbm)
boost <- gbm(CTR ~ ., data = train, distribution = "gaussian", n.trees = 100,
             interaction.depth = 2, shrinkage = 0.01)
boost_predictions <- predict(boost, test, n.trees = 100)
boost_rmse <- sqrt(mean((boost_predictions - test$CTR)^2))
```

- F. Tuned Boosting and XGBoost:

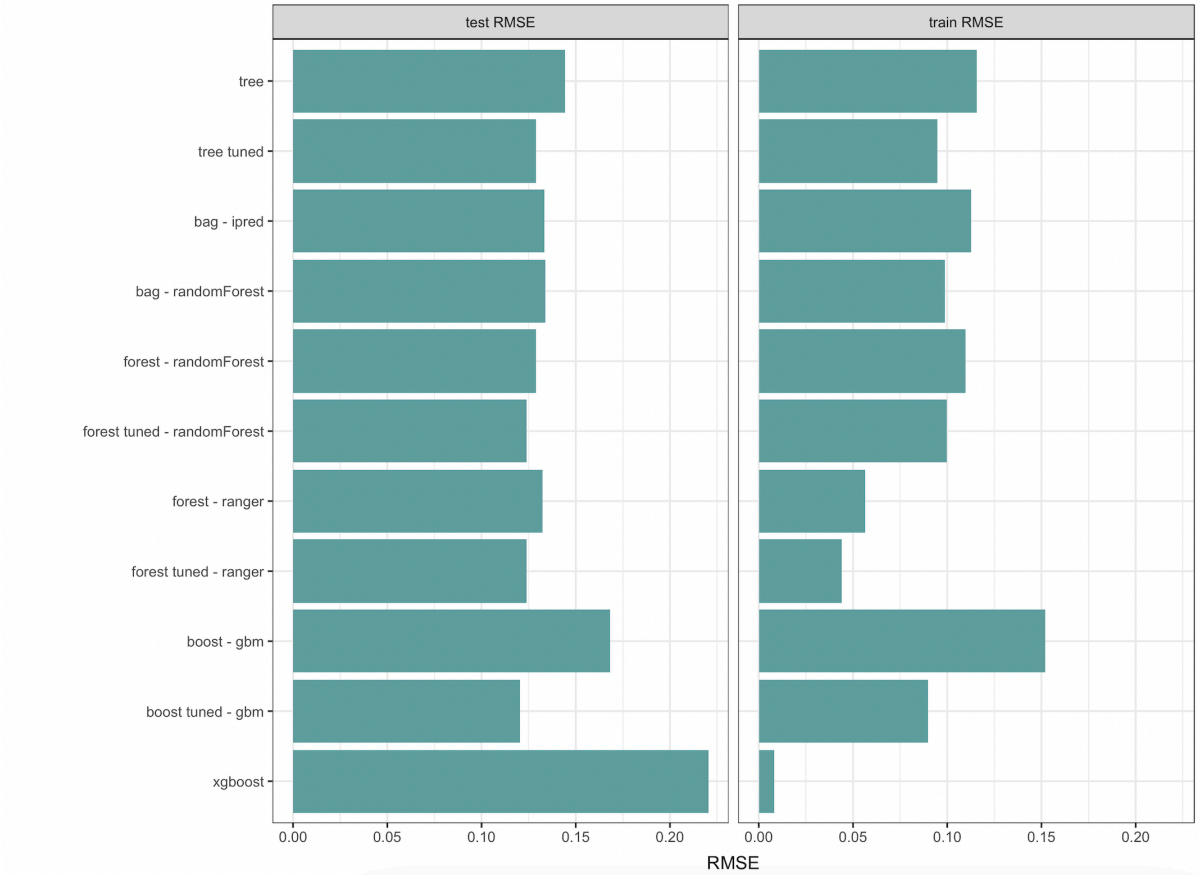
```
library(caret)
tuned_gbm <- train(CTR ~ ., data = train, method = "gbm", trControl = trControl, tuneGri
d = tuneGrid)
xgb_model <- xgboost(data = as.matrix(train %>% select(-CTR)), label = train$CTR,
                     nrounds = 10000, early_stopping_rounds = 100, verbose = FALSE)
```

7. Results and Insights

- Performance Summary:

Model	Train RMSE	Test RMSE
Decision Tree	0.116	0.144
Tuned Decision Tree	0.095	0.129
Random Forest	0.110	0.129
Bagging	0.113	0.133
Gradient Boosting (GBM)	0.090	0.120
Tuned Gradient Boosting	0.090	0.120
XGBoost	0.008	0.220

- Insights:
  - Tuned Boosting and XGBoost models showed the best performance.
  - XGBoost exhibited overfitting with the lowest training RMSE but higher test RMSE.
  - Using VIF-selected important variables improved prediction accuracy.
- Key Visualizations


test RMSE
train RMSE
tree
tree tuned
bag - ipred
bag - randomForest
forest - randomForest
forest tuned - randomForest
forest - ranger
forest tuned - ranger
boost - gbm
boost tuned - gbm
xgboost
0.00
0.05
0.10
0.15
0.20
RMSE

```
library(ggplot2)
# CTR vs Targeting Score
ggplot(data = analysis, aes(x = targeting_score, y = CTR)) +
  geom_point(alpha = 0.5, color = "steelblue") +
  geom_smooth(method = "lm", se = FALSE, color = "darkred") +
  labs(title = "CTR vs Targeting Score",
       x = "Targeting Score",
       y = "Click-Through Rate (CTR)") +
  theme_minimal()

# RMSE Comparison Plot
rmse_df <- data.frame(
  Model = c("Decision Tree", "Tuned Tree", "Random Forest", "Bagging", "GBM", "XGBoos
t"),
  RMSE = c(0.144, 0.129, 0.129, 0.133, 0.120, 0.220)
)

ggplot(rmse_df, aes(x = reorder(Model, RMSE), y = RMSE)) +
  geom_bar(stat = "identity", fill = "skyblue") +
  geom_text(aes(label = round(RMSE, 3)), vjust = -0.3) +
  labs(title = "Test RMSE by Model",
       x = "Model",
       y = "Test RMSE") +
  theme_minimal()

# Feature Importance (requires final_model)
importance_matrix <- xgb.importance(model = final_model)
xgb.plot.importance(importance_matrix, top_n = 10)
```

9. Model Selection - Tuned GBM Boosting VS. XG BOOSTING

- I selected two models to compare their RMSE results on Kaggle.
- Upon submission, the best performance was achieved with XGBoost using a reduced set of variables.
- Therefore, the final model is XGBoost with 5 key variables:
- Tuned models using VIF-selected important variables: targeting_score, visual_appeal,contextual_relevance, headline_length,cta_strength.
- This approach balanced performance and complexity, resulting in a more accurate and efficient model.

```
# data cleaning and preparation
trt = designTreatmentsZ(dframe = analysis, varlist = names(analysis)[2:6])
#Select the variables that are either clean numeric variables or leveled dummy variable
s.
newvars = trt$scoreFrame[trt$scoreFrame$code%in% c('clean','lev'),'varName']
#Prepare the training and testing dataset using the treatment plan.
train_input = prepare(treatmentplan = trt,
                      dframe = analysis,
                      varRestriction = newvars)
test_input = prepare(treatmentplan = trt,
                     dframe = scoring,
                     varRestriction = newvars)
head(train_input)
  targeting_score visual_appeal contextual_relevance headline_length cta_strength
1               4      1.118593                    0             73            0
2               3      4.890355                    0             93            3
3               3      4.744698                    1             96            3
4               6      4.838057                    0             93            5
5               3      1.358385                    0             79            3
6               3      6.288383                    0             92            5
library(xgboost)
set.seed(617)
#Create a DMatrix for training data
dtrain <- xgb.DMatrix(data = as.matrix(train_input), label = analysis$CTR)

# Define the hyperparameters for the XGBoost model
param <- list(
objective = "reg:squarederror",
eta = 0.01,                  # learning rate
max_depth = 6,               # max depth of trees
subsample = 0.8,             # sample rate
colsample_bytree = 0.8,      # column sample rate
lambda = 1,                  # L2 regularization
alpha = 0                    # L1
 )

tune_nrounds <- xgb.cv(
params = param,               # Hyperparameters defined above
data = dtrain,                # Training data in DMatrix format
nrounds = 5000,               # Maximum boosting rounds
nfold = 10,                   # 10-fold cross-validation
early_stopping_rounds = 50,   # Early stopping to prevent overfitting
metrics = "rmse",             # Root Mean Squared Error as the evaluation metric
verbose = 1                   # Display progress during training
)
```

10. Challenges and Missteps

- Initial removal of null values excessively reduced data size.
- Overfitting observed in simpler models with reduced variables.
- Hybrid stepwise selection was computationally intensive but yielded significant insights.

11. Conclusions and Limitations

- Conclusions:
- The tuned boosting model achieved the best prediction accuracy.
- Key variables identified by VIF (`targeting_score` , `visual_appeal` , `contextual_relevance` , `headline_length` , `cta_strength`) were instrumental in improving results.
- XGBoost demonstrated strong performance but risked overfitting.
- Limitations:
- RMSE results depend on data quality and the choice of features.
- Kaggle evaluations highlighted areas for further tuning.

12. Recommendations for Improvement

- Increase the dataset size to improve model generalizability.
- Experiment with additional hyperparameter tuning for XGBoost.
- Explore ensemble methods combining tuned boosting and random forest.